

Part 6: Written Questions (10 marks)

Admission Number: C027-01-0543/2024

Question 1: Design Decisions (2 marks)

- (i) My admission number is **C027-01-0543/2024**. The last digit (3) means I created $5 + 3 = 8$ gigs. The xxxx part (0543) is odd, so my categories are **["Marketing", "Data", "Consulting"]**. The first two digits (05) are less than 10, so my currency is **KES**.
- (ii) I included the status field to track whether a gig is available, underway, or finished. The three possible statuses are **"Open"** (freelancers can apply), **"In Progress"** (a freelancer is working on it), and **"Closed"** (completed or no longer available).
-

Question 2: Validation (2 marks)

- (i) I used `gt=0` (greater than zero) on the budget field so that only positive amounts are accepted. This prevents invalid entries like zero or negative budgets, which don't make sense for a paid gig.
- (ii) I used a regex pattern `^(Marketing|Data|Consulting)$` to restrict the category to only those three values. If a user submits something like "Design" or "Engineering", FastAPI automatically returns a **422 validation error** before the data reaches the endpoint.
-

Question 3: Error Handling (2 marks)

- (i) If someone requests a gig with an ID that doesn't exist, the API returns **HTTP 404 (Not Found)** with the message "Gig not found".
- (ii) If someone sends invalid data (e.g., negative budget or wrong category), FastAPI returns **HTTP 422 (Unprocessable Entity)** with details about which fields failed validation.
-

Question 4: Search and Filtering (2 marks)

- (i) The `/gigs/search?q=` endpoint matches the query against gig titles (case-insensitive). If no gig matches, it returns an **empty list []** — which is the correct behaviour, since a search with no results isn't an error.

(ii) The GET /gigs endpoint accepts optional category, min_budget, and max_budget parameters. Filters are applied sequentially, so users can combine any or all of them — for example, all Marketing gigs under 30,000 KES.

Question 5: Code Walkthrough (2 marks)

(i) The PUT /gigs/{gig_id} endpoint loops through the database looking for a matching ID. If found, it updates only the fields provided (budget and/or status) and returns the updated gig. If no match is found, it raises **HTTP 404** with "Gig not found".

(ii) The POST /gigs endpoint accepts a GigCreate body, assigns a new ID using a global counter, sets status to "Open" by default, adds the gig to the list, and returns it with **HTTP 201 (Created)**. Pydantic handles all input validation before the code runs.